Le développement logiciel à l'ère de l'IA

Cours 3 — Le développement agentique : refermer la boucle

Pierre Sutra

Télécom SudParis

version 1.0

Refermer la boucle

D'un modèle qui suggère à un collaborateur qui agit.

- cours 1 et 2 : un assistant, puis des **outils** ; mais c'est encore vous qui relancez chaque étape
- aujourd'hui : l'**agent** planifie, agit, observe et recommence seul, jusqu'à ce que la tâche soit faite
- votre rôle passe d'*opérateur* à *délégant et vérificateur*

Refermer la boucle

D'un modèle qui suggère à un collaborateur qui agit.

- cours 1 et 2 : un assistant, puis des **outils** ; mais c'est encore vous qui relancez chaque étape
- aujourd'hui : l'**agent** planifie, agit, observe et recommence seul, jusqu'à ce que la tâche soit faite
- votre rôle passe d'*opérateur* à *délégant et vérificateur*

la question centrale : **quelle part du contrôle garder ?**

1. De l'assistant à l'agent
2. La boucle agentique
3. Du *harness* à l'agent de code
4. Rappels sur le développement logiciel
5. Le développement agentique : autonomie et pratique
6. Gérer la fenêtre de contexte
7. Étendre et guider l'agent : AGENTS.md et *skills*

user

Organise un séjour de 3 jours à Lisbonne dans deux mois, avec un budget de 500 €. Je veux quelque chose de pas cher, avec des horaires d'aller et de retour pratiques.

- le *harness* exécutait *un* appel à `search_flights`, et rendait la réponse
- pour le reste du séjour, c'était à *vous* de relancer le modèle à chaque étape : « cherche un hôtel », « vérifie le budget », « écris le plan »

user

Organise un séjour de 3 jours à Lisbonne dans deux mois, avec un budget de 500 €. Je veux quelque chose de pas cher, avec des horaires d'aller et de retour pratiques.

- le *harness* exécutait *un* appel à `search_flights`, et rendait la réponse
- pour le reste du séjour, c'était à *vous* de relancer le modèle à chaque étape : « cherche un hôtel », « vérifie le budget », « écris le plan »
- la boucle interne est déjà automatique : déclarer → exécuter → renvoyer le résultat
- reste manuelle la boucle *externe* : choisir l'étape suivante, et décider que la tâche est finie

user

Organise un séjour de 3 jours à Lisbonne dans deux mois, avec un budget de 500 €. Je veux quelque chose de pas cher, avec des horaires d'aller et de retour pratiques.

- le *harness* exécutait *un* appel à `search_flights`, et rendait la réponse
- pour le reste du séjour, c'était à *vous* de relancer le modèle à chaque étape : « cherche un hôtel », « vérifie le budget », « écris le plan »
- la boucle interne est déjà automatique : déclarer → exécuter → renvoyer le résultat
- reste manuelle la boucle *externe* : choisir l'étape suivante, et décider que la tâche est finie

et si le *harness* ne rendait la main qu'une fois la tâche finie ?

- un **agent** = un LLM + des **outils** + une **boucle** (+ une **mémoire**), le tout dans un *harness*[†]

†. L. Weng. *LLM Powered Autonomous Agents*↗. Lil'Log. 2023.

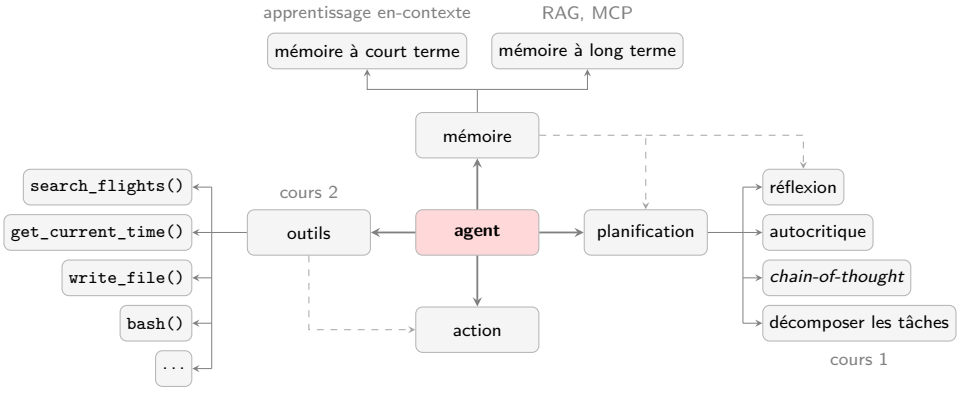
- un **agent** = un LLM + des **outils** + une **boucle** (+ une **mémoire**), le tout dans un *harness*[†]
- un LLM *génère* ; un agent *fait* : avec des outils, en itérant, sur plusieurs étapes qu'il enchaîne seul

†. L. Weng. *LLM Powered Autonomous Agents*↗. Lil'Log. 2023.

- un **agent** = un LLM + des **outils** + une **boucle** (+ une **mémoire**), le tout dans un *harness*[†]
- un LLM *génère* ; un agent *fait* : avec des outils, en itérant, sur plusieurs étapes qu'il enchaîne seul
- ce n'est pas un modèle plus intelligent : c'est l'*infrastructure autour* du modèle qui lui donne la capacité d'agir

†. L. Weng. *LLM Powered Autonomous Agents*↗. Lil'Log. 2023.

Vue d'ensemble : un agent autonome



1. De l'assistant à l'agent
2. La boucle agentique
3. Du *harness* à l'agent de code
4. Rappels sur le développement logiciel
5. Le développement agentique : autonomie et pratique
6. Gérer la fenêtre de contexte
7. Étendre et guider l'agent : AGENTS.md et *skills*

La boucle elle-même : un modèle sans état, un *harness* avec état

- avant tout outil, un agent est une **boucle autour d'un appel au modèle**

une boucle de conversation, une trentaine de lignes avec Ollama

```python
# la mémoire de l'agent
messages = [{'role': 'system', 'content': SYSTEM_PROMPT}]

while True:
    user_input = input("\nYou: ")      # 1. lire
    if user_input.lower() in ['exit', 'quit']:
        break  # arrêt : l'humain décide


    response = ollama.chat(model=MODEL,
        messages=messages)             # 3. tout renvoyer
    agent_message = response['message']
    print(agent_message['content'])    # 4. afficher
```

→ démo : `demo_agent1.py`

La boucle elle-même : un modèle sans état, un *harness* avec état

- avant tout outil, un agent est une **boucle autour d'un appel au modèle**
- le modèle est **sans état** (cours 2) : à chaque tour, on lui renvoie tout

une boucle de conversation, une trentaine de lignes avec Ollama

```
# la mémoire de l'agent
messages = [{'role': 'system', 'content': SYSTEM_PROMPT}]

while True:
    user_input = input("\nYou: ")      # 1. lire
    if user_input.lower() in ['exit', 'quit']:
        break  # arrêt : l'humain décide


    response = ollama.chat(model=MODEL,
        messages=messages)             # 3. tout renvoyer
    agent_message = response['message']
    print(agent_message['content'])    # 4. afficher
```

→ démo : `demo_agent1.py`

La boucle elle-même : un modèle sans état, un *harness* avec état

- avant tout outil, un agent est une **boucle autour d'un appel au modèle**
- le modèle est **sans état** (cours 2) : à chaque tour, on lui renvoie tout
- **la mémoire, c'est la liste `messages`**, tenue par le *harness*

une boucle de conversation, une trentaine de lignes avec Ollama

```python
# la mémoire de l'agent
messages = [{'role': 'system', 'content': SYSTEM_PROMPT}]

while True:
    user_input = input("\nYou: ")      # 1. lire
    if user_input.lower() in ['exit', 'quit']:
        break  # arrêt : l'humain décide
    messages.append({'role': 'user',
        'content': user_input})        # 2. empiler
    response = ollama.chat(model=MODEL,
        messages=messages)             # 3. tout renvoyer
    agent_message = response['message']
    print(agent_message['content'])    # 4. afficher
    messages.append(agent_message)     # 5. retenir
```

→ démo : `demo_agent1.py`

- les conséquences de cette mémoire :
  - chaque tour renvoie tout : le contexte grandit à chaque tour, et le coût de chaque tour avec lui (on y reviendra plus tard)
  - le *harness* peut *modifier* cette mémoire : la tronquer, la résumer (la compaction, section 6), ou y injecter des messages (les résultats d'outils)

- les conséquences de cette mémoire :
  - chaque tour renvoie tout : le contexte grandit à chaque tour, et le coût de chaque tour avec lui (on y reviendra plus tard)
  - le *harness* peut *modifier* cette mémoire : la tronquer, la résumer (la compaction, section 6), ou y injecter des messages (les résultats d'outils)
- mais cette boucle n'est qu'un chatbot (une REPL, *read–eval–print loop*) :
  - chaque itération attend un `input()` humain
  - seul l'humain décide d'arrêter
  - le modèle sait parler, mais pas agir, et ne fait jamais deux pas de suite

- les conséquences de cette mémoire :
  - chaque tour renvoie tout : le contexte grandit à chaque tour, et le coût de chaque tour avec lui (on y reviendra plus tard)
  - le *harness* peut *modifier* cette mémoire : la tronquer, la résumer (la compaction, section 6), ou y injecter des messages (les résultats d'outils)
- mais cette boucle n'est qu'un chatbot (une REPL, *read–eval–print loop*) :
  - chaque itération attend un `input()` humain
  - seul l'humain décide d'arrêter
  - le modèle sait parler, mais pas agir, et ne fait jamais deux pas de suite
- la suite de la section retire l'humain de la boucle interne, en deux temps :
  1. donner des outils au modèle (cours 2)
  2. laisser les résultats des outils, et non plus `input()`, déclencher l'itération suivante

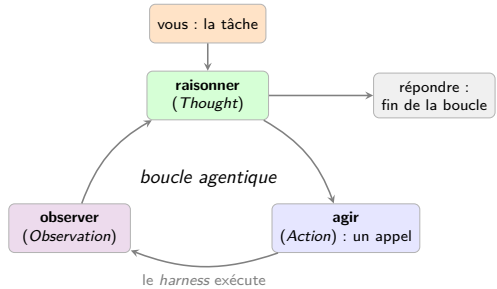
Rappel : un appel d'outil

- le *harness* passe au modèle les schémas des outils ; le modèle *déclare* un appel en JSON ; le *harness* le valide, l'exécute, et empile le résultat dans un message du rôle ***tool***
- l'origine de Claude Code (cours 2) : Boris Cherny donne à Sonnet 3.5 *un seul* outil `bash`, et le modèle écrit un AppleScript pour trouver la musique qu'il écoute
- **un outil général et une boucle, c'est déjà un agent**

Rappel : un appel d'outil

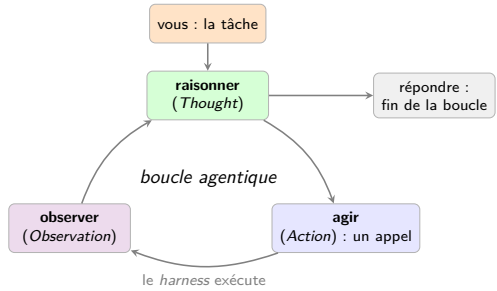
- le *harness* passe au modèle les schémas des outils ; le modèle *déclare* un appel en JSON ; le *harness* le valide, l'exécute, et empile le résultat dans un message du rôle ***tool***
- l'origine de Claude Code (cours 2) : Boris Cherny donne à Sonnet 3.5 *un seul* outil `bash`, et le modèle écrit un AppleScript pour trouver la musique qu'il écoute
- **un outil général et une boucle, c'est déjà un agent**
- un agent de code n'a besoin que de peu d'outils, et très généraux :
  - `bash`
  - lire, écrire, modifier un fichier
  - chercher dans le code (`grep`)
  - chercher et lire sur le web

- **ReAct** † : le modèle alterne
  - *penser* (*Thought*)
  - *agir* (*Action* = un appel d'outil)
  - *lire le résultat de l'outil* (*Observation*)



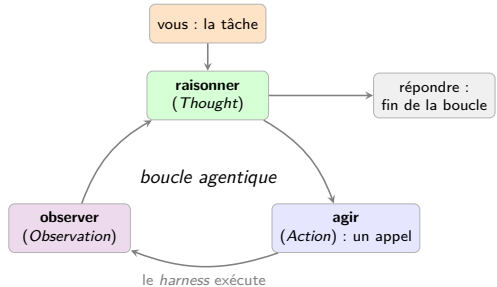
†. S. Yao et al. « ReAct : Synergizing Reasoning and Acting in Language Models »↗. *ICLR*. 2023.

- **ReAct** † : le modèle alterne
  - *penser* (*Thought*)
  - *agir* (*Action* = un appel d'outil)
  - *lire le résultat de l'outil* (*Observation*)
- le *chain-of-thought* du cours 1, **plus des outils** : vérifier un fait au lieu de le supposer



†. S. Yao et al. « ReAct : Synergizing Reasoning and Acting in Language Models »↗. *ICLR*. 2023.

- **ReAct** † : le modèle alterne
  - *penser* (*Thought*)
  - *agir* (*Action* = un appel d'outil)
  - *lire le résultat de l'outil* (*Observation*)
- le *chain-of-thought* du cours 1, **plus des outils** : vérifier un fait au lieu de le supposer
- à chaque tour (cours 2) : **un appel**, la boucle continue ; **une réponse**, elle s'arrête



†. S. Yao et al. « ReAct : Synergizing Reasoning and Acting in Language Models »↗. *ICLR*. 2023.

Exemple : la tâche de Lisbonne, menée par l'agent

user

Organise un séjour de 3 jours à Lisbonne à la mi-mai, avec un budget de 500 €. Je veux quelque chose de pas cher, avec des horaires d'aller et de retour pratiques.

Thought `Il me faut de vrais vols ; je ne connais ni les prix ni les horaires.`

Action
```
search_flights("Paris", "Lisbon",
        "2026-05-15")
```

Obs. `TAP 1702 06:10 45€ | Iberia 894 09:00 90€`

Exemple : la tâche de Lisbonne, menée par l'agent

user

Organise un séjour de 3 jours à Lisbonne à la mi-mai, avec un budget de 500 €. Je veux quelque chose de pas cher, avec des horaires d'aller et de retour pratiques.

Thought `Il me faut de vrais vols ; je ne connais ni les prix ni les horaires.`

Action `search_flights("Paris", "Lisbon", "2026-05-15")`

Obs. `TAP 1702 06:10 45€ | Iberia 894 09:00 90€`

Thought `06:10 est bon marché mais peu pratique ; 09:00 coûte 45€ de plus. Vérifier d'abord le retour et un hôtel.`

Action `search_flights("Lisbon", "Paris", "2026-05-18")`

Obs. `TAP 1705 18:30 60€`

Action `search_hotels("Lisbon", "2026-05-15", nights=3)`

Obs. `Casa Alfama 70€/nuit | Hotel Baixa 110€/nuit`

Exemple : la tâche de Lisbonne, menée par l'agent

user

Organise un séjour de 3 jours à Lisbonne à la mi-mai, avec un budget de 500 €. Je veux quelque chose de pas cher, avec des horaires d'aller et de retour pratiques.

Thought `Il me faut de vrais vols ; je ne connais ni les prix ni les horaires.`

Action `search_flights("Paris", "Lisbon", "2026-05-15")`

Obs. `TAP 1702 06:10 45€ | Iberia 894 09:00 90€`

Thought `06:10 est bon marché mais peu pratique ; 09:00 coûte 45€ de plus. Vérifier d'abord le retour et un hôtel.`

Action `search_flights("Lisbon", "Paris", "2026-05-18")`

Obs. `TAP 1705 18:30 60€`

Action `search_hotels("Lisbon", "2026-05-15", nights=3)`

Obs. `Casa Alfama 70€/nuit | Hotel Baixa 110€/nuit`

Thought `90 + 60 + 3×70 = 360€ ; il reste 140€ pour les repas et les transports.`

Action `save_plan("lisbon.md", "...")`

Obs. `ok`

Exemple : la tâche de Lisbonne, menée par l'agent

user

Organise un séjour de 3 jours à Lisbonne à la mi-mai, avec un budget de 500 €. Je veux quelque chose de pas cher, avec des horaires d'aller et de retour pratiques.

Thought Il me faut de vrais vols ; je ne connais ni les prix ni les horaires.

Action `search_flights("Paris", "Lisbon", "2026-05-15")`

Obs. TAP 1702 06:10 45€ | Iberia 894 09:00 90€

Thought 06:10 est bon marché mais peu pratique ; 09:00 coûte 45€ de plus. Vérifier d'abord le retour et un hôtel.

Action `search_flights("Lisbon", "Paris", "2026-05-18")`

Obs. TAP 1705 18:30 60€

Action `search_hotels("Lisbon", "2026-05-15", nights=3)`

Obs. Casa Alfama 70€/nuit | Hotel Baixa 110€/nuit

Thought 90 + 60 + 3×70 = 360€ ; il reste 140€ pour les repas et les transports.

Action `save_plan("lisbon.md", "...")`

Obs. ok

Thought Réserver dépenserait l'argent de l'utilisateur : il faut son accord.

Answer Plan enregistré dans `lisbon.md` (360€ + 140€ sur place). Je réserve ?

Ce que montre la trace

- personne n'a relancé le modèle entre deux étapes : il a choisi chaque action à partir de l'observation précédente
- l'arbitrage « pas cher contre pratique » se fait *sur des preuves* (le RAG du cours 2), pas de mémoire
- l'agent s'arrête de lui-même (il répond), et s'arrête *avant* l'étape irréversible : la réservation

La boucle doit toujours pouvoir s'arrêter

Les conditions d'arrêt

- **but atteint** : le modèle répond au lieu d'appeler un outil (succès, les tests passent)
- **nombre maximal d'étapes** : le budget est épuisé (étapes, *tokens*, temps)
- **interruption** par l'utilisateur : toujours possible
- pour une grosse tâche : planifier, puis exécuter (le mode plan, section 5)

La boucle doit toujours pouvoir s'arrêter

Les conditions d'arrêt

- **but atteint** : le modèle répond au lieu d'appeler un outil (succès, les tests passent)
- **nombre maximal d'étapes** : le budget est épuisé (étapes, *tokens*, temps)
- **interruption** par l'utilisateur : toujours possible
- pour une grosse tâche : planifier, puis exécuter (le mode plan, section 5)

Détecter le blocage, demander de l'aide

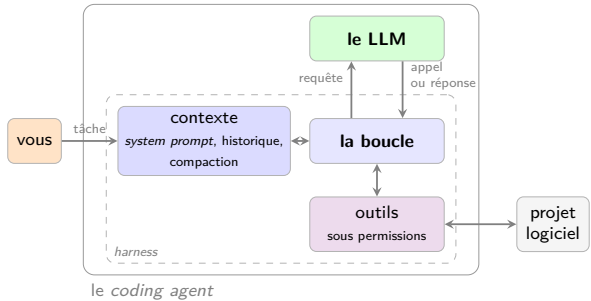
- les symptômes : oscillation (la même action répétée), erreurs d'outils à répétition, boucles sans progrès ; le *harness* doit les détecter, puis demander, escalader ou arrêter
- côté modèle : l'**abstention contrôlée** (cours 1), un agent qui dit « il me manque X, dois-je supposer Y ? » au lieu de deviner
- plus important pour un agent que pour un assistant : une erreur n'est plus une mauvaise *phrase*, mais une mauvaise *suite d'actions*

1. De l'assistant à l'agent
2. La boucle agentique
3. Du *harness* à l'agent de code
4. Rappels sur le développement logiciel
5. Le développement agentique : autonomie et pratique
6. Gérer la fenêtre de contexte
7. Étendre et guider l'agent : AGENTS.md et *skills*

- un **agent de code** (*coding agent*, on dit aussi *AI coding assistant*) est un agent dont l'environnement est un projet logiciel : il lit le code, le modifie, l'exécute et le teste, en boucle, jusqu'à ce que la tâche soit faite

- un **agent de code** (*coding agent*, on dit aussi *AI coding assistant*) est un agent dont l'environnement est un projet logiciel : il lit le code, le modifie, l'exécute et le teste, en boucle, jusqu'à ce que la tâche soit faite
- c'est **un LLM, plus un *harness*** spécialisé pour le développement :

la boucle	raisonner, agir, observer, jusqu'à l'arrêt (section 2)
outils	fichiers, `bash`, recherche dans le code et sur le web
contexte	ce que le modèle reçoit à chaque appel : *system prompt*, historique
permissions	ce qu'il fait sans demander
compaction	ce qu'on fait quand la conversation dépasse la fenêtre (section 6)
prompt	les instructions système (cours 1 : persona, *template*...)



le *coding agent*

- un agent de code n'a rien de magique †
- `demo_agent2.py`, c'est la boucle, quatre outils (lire, écrire, exécuter un fichier, chercher sur Wikipédia) et un *system prompt*

†. M. Eric. *The Emperor Has No Clothes : How to Code Claude Code in 200 Lines of Code*. mihaileric.com. 2025.

user

Quelle approximation de π obtient-on avec les 1 000 premiers termes de la série de Leibniz ?

→ démo : `demo_agent2.py`

user

Quelle approximation de π obtient-on avec les 1 000 premiers termes de la série de Leibniz ?

Thought Je vais calculer l'approximation de π en utilisant les 1 000 premiers termes de la série de Leibniz.

Action `wikipedia_search(query="série de Leibniz", lang="fr")`

Obs. `# Formule de Leibniz pour $\pi$ [...] ${\frac {\pi }{4}}=1-{\frac {1}{3}}+\cdots$ [...]`

Thought Je vais calculer la somme des 1 000 premiers termes [...], puis multiplier par 4 pour obtenir π.

Action `write_file(path="leibniz_series.py", content= "[...] for k in range(1000): [...] pi_approx = 4 * sum_leibniz [...]")`

Obs. `Wrote 406 characters to /tmp/agent/leibniz_series.py`

Thought Je vais exécuter le script pour obtenir le résultat.

Action `run_python(path="/tmp/agent/leibniz_series.py")`

Obs. `exit code: 0 Approximation de π [...] : 3.140592653839794`

Answer L'approximation de π obtenue avec les 1 000 premiers termes de la série de Leibniz est d'environ 3,14059. [...] la valeur exacte de π est d'environ 3,14159.

Le même modèle, des agents de code différents

- le *même* LLM se comporte tout autrement selon l'agent de code qui l'entoure
- l'agent de code décide : à quel point il est entreprenant, à quoi il a accès, comment il gère son contexte
- il existe aussi des *harness* spécialisés pour d'autres tâches, par exemple les mathématiques de recherche ou les systèmes multi-agents (voir cours 4)

quelques agents de code

Claude Code	CLI
opencode	CLI
pi	CLI
Hermes	CLI
agy	CLI
aider	CLI
Cursor	GUI (fork de VS Code)
Copilot (mode agent)	GUI (VS Code, GitHub)

Le même modèle, des agents de code différents

- le *même* LLM se comporte tout autrement selon l'agent de code qui l'entoure
- l'agent de code décide : à quel point il est entreprenant, à quoi il a accès, comment il gère son contexte
- il existe aussi des *harness* spécialisés pour d'autres tâches, par exemple les mathématiques de recherche ou les systèmes multi-agents (voir cours 4)

→ projet : `README.md` : le projet `wikidiff`, à faire avec opencode v2.2.5+

quelques agents de code

Claude Code	CLI
opencode	CLI
pi	CLI
Hermes	CLI
agy	CLI
aider	CLI
Cursor	GUI (fork de VS Code)
Copilot (mode agent)	GUI (VS Code, GitHub)

1. De l'assistant à l'agent
2. La boucle agentique
3. Du *harness* à l'agent de code
4. Rappels sur le développement logiciel
5. Le développement agentique : autonomie et pratique
6. Gérer la fenêtre de contexte
7. Étendre et guider l'agent : AGENTS.md et *skills*

- un logiciel se fait **en équipe**, et des agents y participent déjà

- un logiciel se fait **en équipe**, et des agents y participent déjà
- des **sources** (le code de l'équipe) et des **dépendances** (le code des autres)

- un logiciel se fait **en équipe**, et des agents y participent déjà
- des **sources** (le code de l'équipe) et des **dépendances** (le code des autres)
- un **dépôt** garde les sources et l'historique de chaque changement

- un logiciel se fait **en équipe**, et des agents y participent déjà
- des **sources** (le code de l'équipe) et des **dépendances** (le code des autres)
- un **dépôt** garde les sources et l'historique de chaque changement
- chacun travaille sur sa **copie locale**, puis partage ses changements, une fois prêts

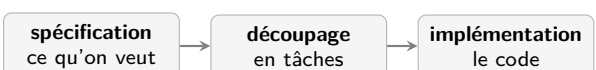
- un logiciel se fait **en équipe**, et des agents y participent déjà
- des **sources** (le code de l'équipe) et des **dépendances** (le code des autres)
- un **dépôt** garde les sources et l'historique de chaque changement
- chacun travaille sur sa **copie locale**, puis partage ses changements, une fois prêts
- les grandes étapes d'une évolution :

spécification
ce qu'on veut

- un logiciel se fait **en équipe**, et des agents y participent déjà
- des **sources** (le code de l'équipe) et des **dépendances** (le code des autres)
- un **dépôt** garde les sources et l'historique de chaque changement
- chacun travaille sur sa **copie locale**, puis partage ses changements, une fois prêts
- les grandes étapes d'une évolution :



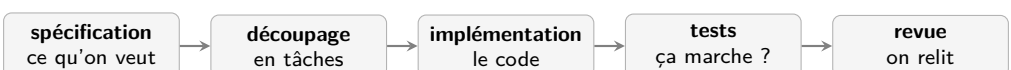
- un logiciel se fait **en équipe**, et des agents y participent déjà
- des **sources** (le code de l'équipe) et des **dépendances** (le code des autres)
- un **dépôt** garde les sources et l'historique de chaque changement
- chacun travaille sur sa **copie locale**, puis partage ses changements, une fois prêts
- les grandes étapes d'une évolution :



- un logiciel se fait **en équipe**, et des agents y participent déjà
- des **sources** (le code de l'équipe) et des **dépendances** (le code des autres)
- un **dépôt** garde les sources et l'historique de chaque changement
- chacun travaille sur sa **copie locale**, puis partage ses changements, une fois prêts
- les grandes étapes d'une évolution :



- un logiciel se fait **en équipe**, et des agents y participent déjà
- des **sources** (le code de l'équipe) et des **dépendances** (le code des autres)
- un **dépôt** garde les sources et l'historique de chaque changement
- chacun travaille sur sa **copie locale**, puis partage ses changements, une fois prêts
- les grandes étapes d'une évolution :



- **version** : l'état de tous les fichiers du logiciel à un moment donné ; chaque changement produit une nouvelle version (v1, v2, v3...)

- **version** : l'état de tous les fichiers du logiciel à un moment donné ; chaque changement produit une nouvelle version (v1, v2, v3...)
- ***commit*** : l'enregistrement d'une nouvelle version, avec son auteur, sa date, et un message qui explique le changement

- **version** : l'état de tous les fichiers du logiciel à un moment donné ; chaque changement produit une nouvelle version (v1, v2, v3…)
- ***commit*** : l'enregistrement d'une nouvelle version, avec son auteur, sa date, et un message qui explique le changement
- ***diff*** : ce qui a changé entre deux *commits*, ligne par ligne : les lignes retirées (−) et ajoutées (+)

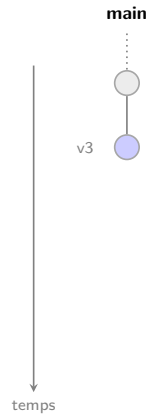
un *commit*, et son *diff* avec le *commit* précédent

```
commit 3f2a9c1  Alice, le 12 mars
    Arrondit le total au centime
--- a/prix.py  +++ b/prix.py
 def total(prix):
-    return sum(prix)
+    return round(sum(prix), 2)
```

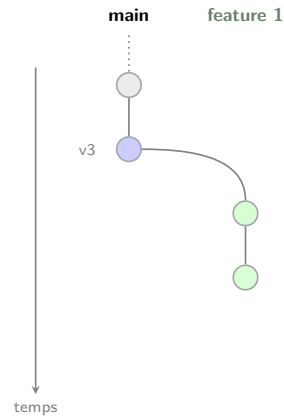
- **version** : l'état de tous les fichiers du logiciel à un moment donné ; chaque changement produit une nouvelle version (v1, v2, v3…)
- ***commit*** : l'enregistrement d'une nouvelle version, avec son auteur, sa date, et un message qui explique le changement
- ***diff*** : ce qui a changé entre deux *commits*, ligne par ligne : les lignes retirées (−) et ajoutées (+)
- **git**, le gestionnaire de versions le plus répandu, garde toutes les versions : on peut revenir à celle qui marchait

un *commit*, et son *diff* avec le *commit* précédent

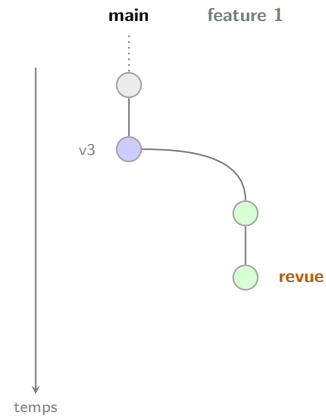
```
commit 3f2a9c1  Alice, le 12 mars
    Arrondit le total au centime
--- a/prix.py  +++ b/prix.py
 def total(prix):
-    return sum(prix)
+    return round(sum(prix), 2)
```


main
v3
temps

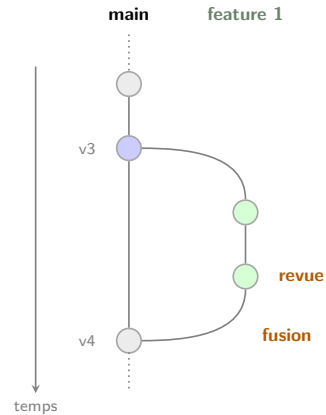
- **branche** : une copie de l'historique, pour essayer sans toucher à `main`, la version de référence



- **branche** : une copie de l'historique, pour essayer sans toucher à `main`, la version de référence
- **revue de code** : avant d'accepter une branche, quelqu'un d'autre relit son *diff* ; il l'accepte, demande des corrections, ou la refuse



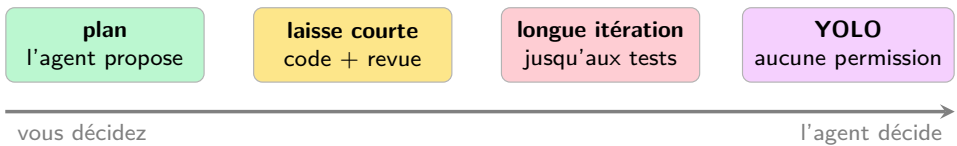
- **branche** : une copie de l'historique, pour essayer sans toucher à `main`, la version de référence
- **revue de code** : avant d'accepter une branche, quelqu'un d'autre relit son *diff* ; il l'accepte, demande des corrections, ou la refuse
- **fusion** (*merge*) : ajouter les *commits* de la branche à `main` ; si c'est raté, on jette la branche, et `main` n'a pas bougé



1. De l'assistant à l'agent
2. La boucle agentique
3. Du *harness* à l'agent de code
4. Rappels sur le développement logiciel
5. Le développement agentique : autonomie et pratique
6. Gérer la fenêtre de contexte
7. Étendre et guider l'agent : AGENTS.md et *skills*

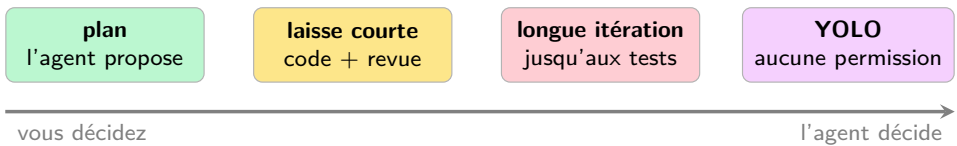
La question centrale : quelle part du contrôle garder ?

- agir donne du pouvoir à l'agent ; le pouvoir demande des limites



La question centrale : quelle part du contrôle garder ?

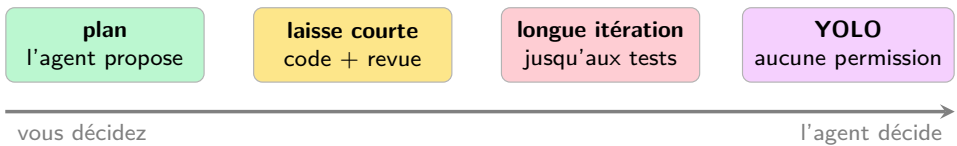
- agir donne du pouvoir à l'agent ; le pouvoir demande des limites



- chaque case du cadran est un **mode** ; on choisit le sien selon trois critères :
  - le **rayon d'impact** : ce qu'une erreur peut casser
  - la **vérifiabilité** : l'agent peut-il vérifier lui-même son travail ?
  - la **réversibilité** : une erreur peut-elle être défaite ?

La question centrale : quelle part du contrôle garder ?

- agir donne du pouvoir à l'agent ; le pouvoir demande des limites



- chaque case du cadran est un **mode** ; on choisit le sien selon trois critères :
  - le **rayon d'impact** : ce qu'une erreur peut casser
  - la **vérifiabilité** : l'agent peut-il vérifier lui-même son travail ?
  - la **réversibilité** : une erreur peut-elle être défaite ?
- et la question miroir : **quand *ne pas* utiliser d'agent ?** par exemple, quand le coût ou la latence d'une boucle ne se justifient pas

- chaque mode est suivi d'un cas d'usage, décrit par une fiche
- pour chaque tâche, les quatre mêmes questions :
  - *quel mode ?*
  - *quand l'agent s'arrête-t-il ?*
  - *quel contexte donner ?*
  - *qu'est-ce qui peut mal tourner ?*

Laisse courte : *prompt* → code → revue



- principe : l'agent écrit le code → vous relisez le code → on itère
- l'agent reste sur des changements atomiques : le rayon d'impact est petit, et chaque revue de code est un point de contrôle
- usage : une tâche simple aux contours bien définie avec une validité simple (e.g., tests unitaires)

Cas d'usage : optimiser les performances

user

`process_data`, dans `src/analytics.py`, est trop lente sur un million de lignes. Mesure-la d'abord, propose le changement, puis implémente-le.

user

`process_data`, dans `src/analytics.py`, est trop lente sur un million de lignes. Mesure-la d'abord, propose le changement, puis implémente-le.

---

mode laisse courte : une optimisation par itération, relue

Cas d'usage : optimiser les performances

user

`process_data`, dans `src/analytics.py`, est trop lente sur un million de lignes. Mesure-la d'abord, propose le changement, puis implémente-le.

mode	laisse courte : une optimisation par itération, relue
fini quand	un *benchmark* s'améliore *et* les tests passent toujours : deux oracles, la vitesse et la correction

Cas d'usage : optimiser les performances

user

`process_data`, dans `src/analytics.py`, est trop lente sur un million de lignes. Mesure-la d'abord, propose le changement, puis implémente-le.

mode	laisse courte : une optimisation par itération, relue
fini quand	un *benchmark* s'améliore *et* les tests passent toujours : deux oracles, la vitesse et la correction
contexte	une taille d'entrée réaliste (la fonction `process_data`), et un *benchmark* que vous comprenez aisément

Cas d'usage : optimiser les performances

user

`process_data`, dans `src/analytics.py`, est trop lente sur un million de lignes. Mesure-la d'abord, propose le changement, puis implémente-le.

mode	laisse courte : une optimisation par itération, relue
fini quand	un *benchmark* s'améliore *et* les tests passent toujours : deux oracles, la vitesse et la correction
contexte	une taille d'entrée réaliste (la fonction `process_data`), et un *benchmark* que vous comprenez aisément
risque	une optimisation qui change le comportement sur cas limites, ou un *benchmark* que l'agent ajuste à son avantage → garder le *benchmark* hors de sa portée

Le mode plan : réfléchir avant d'agir



- l'agent propose un plan, vous le relisez, puis il l'exécute
- il lit les fichiers, cherche, synthétise une stratégie, *sans écrire de code*
- cela évite les changements irréversibles avant que vous n'ayez validé l'approche
- usage : tâches complexes, refactorisations sur plusieurs fichiers, demandes ambiguës

Le mode plan : réfléchir avant d'agir



- l'agent propose un plan, vous le relisez, puis il l'exécute
- il lit les fichiers, cherche, synthétise une stratégie, *sans écrire de code*
- cela évite les changements irréversibles avant que vous n'ayez validé l'approche
- usage : tâches complexes, refactorisations sur plusieurs fichiers, demandes ambiguës

opencode : `Shift+Tab` passe du mode *build* au mode *plan*, et inversement

Cas d'usage : comprendre un code inconnu

user

Comment est gérée l'authentification depuis `user_login.py` ? Dessine son graphe d'appels.

user

Comment est gérée l'authentification depuis `user_login.py` ? Dessine son graphe d'appels.

---

mode lecture seule, ou mode plan : rien n'est écrit, le rayon d'impact est nul

---

user

Comment est gérée l'authentification depuis `user_login.py` ? Dessine son graphe d'appels.

mode	lecture seule, ou mode plan : rien n'est écrit, le rayon d'impact est nul
fini quand	vos questions ont une réponse, avec des références `fichier:ligne` vérifiables

user

Comment est gérée l'authentification depuis `user_login.py` ? Dessine son graphe d'appels.

mode	lecture seule, ou mode plan : rien n'est écrit, le rayon d'impact est nul
fini quand	vos questions ont une réponse, avec des références `fichier:ligne` vérifiables
contexte	laisser l'agent chercher (`grep`, lecture) plutôt que coller des fichiers

user

Comment est gérée l'authentification depuis `user_login.py` ? Dessine son graphe d'appels.

mode	lecture seule, ou mode plan : rien n'est écrit, le rayon d'impact est nul
fini quand	vos questions ont une réponse, avec des références `fichier:ligne` vérifiables
contexte	laisser l'agent chercher (`grep`, lecture) plutôt que coller des fichiers
risque	une explication sûre d'elle, mais fausse (cours 1, hallucinations) → demander des références, en ouvrir quelques-unes

user

Comment est gérée l'authentification depuis `user_login.py` ? Dessine son graphe d'appels.

mode	lecture seule, ou mode plan : rien n'est écrit, le rayon d'impact est nul
fini quand	vos questions ont une réponse, avec des références `fichier:ligne` vérifiables
contexte	laisser l'agent chercher (`grep`, lecture) plutôt que coller des fichiers
risque	une explication sûre d'elle, mais fausse (cours 1, hallucinations) → demander des références, en ouvrir quelques-unes

- le meilleur premier usage d'un agent sur un projet qu'on vient d'ouvrir



- l'agent itère seul, jusqu'au succès
- la condition indispensable : des **résultats vérifiables**, l'agent doit pouvoir tester son propre travail
- usage : les tâches avec un critère de succès clair (les tests passent, la sortie est correcte)

Cas d'usage : corriger un test qui échoue

user

`test_user_registration`, dans `tests/test_auth.py`, échoue avec une `AttributeError`. Trouve la cause et corrige-la.

Cas d'usage : corriger un test qui échoue

user

`test_user_registration`, dans `tests/test_auth.py`, échoue avec une `AttributeError`. Trouve la cause et corrige-la.

---

mode longue itération : le test qui échoue est un oracle tout prêt

---

Cas d'usage : corriger un test qui échoue

user

`test_user_registration`, dans `tests/test_auth.py`, échoue avec une `AttributeError`. Trouve la cause et corrige-la.

mode	longue itération : le test qui échoue est un oracle tout prêt
fini quand	ce test *et le reste de la suite* passent

Cas d'usage : corriger un test qui échoue

user

`test_user_registration`, dans `tests/test_auth.py`, échoue avec une `AttributeError`. Trouve la cause et corrige-la.

mode	longue itération : le test qui échoue est un oracle tout prêt
fini quand	ce test *et le reste de la suite* passent
contexte	le message d'erreur complet, et la commande qui le reproduit ; pas votre hypothèse sur la cause

Cas d'usage : corriger un test qui échoue

user

`test_user_registration`, dans `tests/test_auth.py`, échoue avec une `AttributeError`. Trouve la cause et corrige-la.

mode	longue itération : le test qui échoue est un oracle tout prêt
fini quand	ce test *et le reste de la suite* passent
contexte	le message d'erreur complet, et la commande qui le reproduit ; pas votre hypothèse sur la cause
risque	corriger le symptôme (un `try/except`, un cas particulier) plutôt que la cause → demander la cause en une phrase, avant la correction

- ***Reflexion*** : après un échec, le modèle écrit une critique de *pourquoi* il a échoué ; cette critique est injectée dans la tentative suivante

- ***Reflexion*** : après un échec, le modèle écrit une critique de *pourquoi* il a échoué ; cette critique est injectée dans la tentative suivante
- dans le cours 1, c'était l'exemple avec `is_valid_password`

- ***Reflexion*** : après un échec, le modèle écrit une critique de *pourquoi* il a échoué ; cette critique est injectée dans la tentative suivante
- dans le cours 1, c'était l'exemple avec `is_valid_password`
- ici c'est la suite de tests (pas un humain) qui tourne et les *échecs* sont renvoyés à l'agent

- ***Reflexion*** : après un échec, le modèle écrit une critique de *pourquoi* il a échoué ; cette critique est injectée dans la tentative suivante
- dans le cours 1, c'était l'exemple avec `is_valid_password`
- ici c'est la suite de tests (pas un humain) qui tourne et les *échecs* sont renvoyés à l'agent
- un agent en longue itération, c'est *Reflexion* avec des outils : il lance lui-même les tests, lit les échecs, et recommence

- cours 1 : un jeu de référence pour les *prompts* ; cours 2 : des tests pour la couche outils ; pour un agent, la **suite de tests est l'oracle** qui définit « fini » (la condition d'arrêt « but atteint »)

- cours 1 : un jeu de référence pour les *prompts* ; cours 2 : des tests pour la couche outils ; pour un agent, la **suite de tests est l'oracle** qui définit « fini » (la condition d'arrêt « but atteint »)
- pas d'oracle → l'agent s'arrête quand il *croit* avoir fini → le garder en laisse courte

- cours 1 : un jeu de référence pour les *prompts* ; cours 2 : des tests pour la couche outils ; pour un agent, la **suite de tests est l'oracle** qui définit « fini » (la condition d'arrêt « but atteint »)
- pas d'oracle → l'agent s'arrête quand il *croit* avoir fini → le garder en laisse courte
- un bon oracle (tests, vérificateur de types, *linter*, *benchmark*) → la longue itération devient sûre

- cours 1 : un jeu de référence pour les *prompts* ; cours 2 : des tests pour la couche outils ; pour un agent, la **suite de tests est l'oracle** qui définit « fini » (la condition d'arrêt « but atteint »)
- pas d'oracle → l'agent s'arrête quand il *croit* avoir fini → le garder en laisse courte
- un bon oracle (tests, vérificateur de types, *linter*, *benchmark*) → la longue itération devient sûre
- corollaire : écrire les tests *avant* de déléguer est ce qui rapporte le plus (on appelle cela du *test-driven development)*

Cas d'usage : ajouter une fonctionnalité, les tests d'abord

user

Écris les tests unitaires de la classe Java `MyNewAPI` qui permet de [..]. Ne l'implémente pas.

↓ vous relisez les tests

user

Maintenant, implémente cette classe jusqu'à ce que les tests passent ; ne modifie pas les tests.

Cas d'usage : ajouter une fonctionnalité, les tests d'abord

user

Écris les tests unitaires de la classe Java `MyNewAPI` qui permet de [..]. Ne l'implémente pas.

↓ vous relisez les tests

user

Maintenant, implémente cette classe jusqu'à ce que les tests passent ; ne modifie pas les tests.

---

mode plan, puis longue itération en suivant le plan

Cas d'usage : ajouter une fonctionnalité, les tests d'abord

user

Écris les tests unitaires de la classe Java `MyNewAPI` qui permet de [..]. Ne l'implémente pas.

↓ vous relisez les tests

user

Maintenant, implémente cette classe jusqu'à ce que les tests passent ; ne modifie pas les tests.

mode	plan, puis longue itération en suivant le plan
fini quand	les tests *que vous avez relus avant* passent : l'oracle définit « fini »

Cas d'usage : ajouter une fonctionnalité, les tests d'abord

user

Écris les tests unitaires de la classe Java `MyNewAPI` qui permet de [..]. Ne l'implémente pas.

↓ vous relisez les tests

user

Maintenant, implémente cette classe jusqu'à ce que les tests passent ; ne modifie pas les tests.

mode	plan, puis longue itération en suivant le plan
fini quand	les tests *que vous avez relus avant* passent : l'oracle définit « fini »
contexte	la documentation (ou le prompt) indique comment lancer les tests ; le code existant, pour suivre les conventions d'écriture

Cas d'usage : ajouter une fonctionnalité, les tests d'abord

user

Écris les tests unitaires de la classe Java `MyNewAPI` qui permet de [..]. Ne l'implémente pas.

↓ vous relisez les tests

user

Maintenant, implémente cette classe jusqu'à ce que les tests passent ; ne modifie pas les tests.

mode	plan, puis longue itération en suivant le plan
fini quand	les tests *que vous avez relus avant* passent : l'oracle définit « fini »
contexte	la documentation (ou le prompt) indique comment lancer les tests ; le code existant, pour suivre les conventions d'écriture
risque	l'agent modifie ou affaiblit les tests pour les faire passer → l'interdire dans le *prompt*, vérifier `git diff tests/` à la fin

Le mode YOLO : l'autonomie complète



- l'agent exécute n'importe quelle commande, modifie n'importe quel fichier, sans rien demander
- usage : environnements de confiance, expériences jetables, prototypage rapide
- à éviter : le code en production, les projets sensibles, les dépôts partagés
- le filet de sécurité, même en YOLO : relire avec *git diff* avant d'accepter

Le mode YOLO : l'autonomie complète



- l'agent exécute n'importe quelle commande, modifie n'importe quel fichier, sans rien demander
- usage : environnements de confiance, expériences jetables, prototypage rapide
- à éviter : le code en production, les projets sensibles, les dépôts partagés
- le filet de sécurité, même en YOLO : relire avec *git diff* avant d'accepter

opencode : *Settings* → *Permissions* → *Auto accept*

Cours 1 : l'humain à chaque étape

décrire → générer → relire → tester → affiner

Avec un agent

décrire → **l'agent génère, agit et se teste** → vous relisez le *diff* et les tests

- l'itération *intérieure* est déléguée ; la boucle extérieure (la spécification, la vérification) reste humaine
- les modes d'échec s'adaptent aussi : *spécifications* vagues, contexte du *projet* manquant, appels d'outils hallucinés, sur-génération

- **une tâche, une branche** : une exécution ratée part avec sa branche
- ***commiter* avant de déléguer** : `git diff` montre alors exactement ce que l'agent a changé, et `git restore .` l'annule
- **de petits *commits*** : chacun est un point de contrôle, qu'on relit et qu'on peut défaire
- **lire le *diff*, pas le résumé de l'agent** : le résumé dit ce qu'il *croit* avoir fait

c'est ce qui rend acceptables la longue itération et le mode YOLO :
chaque action devient réversible

À retenir

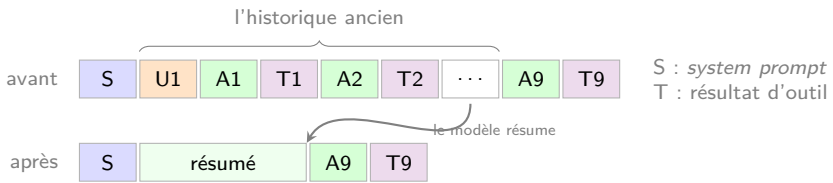
1. **l'autonomie est un cadran**, pas un interrupteur : plan, laisse courte, longue itération, YOLO
2. on le règle selon le **rayon d'impact**, la **vérifiabilité** et la **réversibilité**
3. **l'oracle définit « fini »** : sans tests, laisse courte ; avec de bons tests, longue itération
4. écrire les tests *avant* de déléguer, et vérifier que l'agent ne les a pas modifiés
5. quel que soit le mode, **relire le *diff***

1. De l'assistant à l'agent
2. La boucle agentique
3. Du *harness* à l'agent de code
4. Rappels sur le développement logiciel
5. Le développement agentique : autonomie et pratique
6. Gérer la fenêtre de contexte
7. Étendre et guider l'agent : AGENTS.md et *skills*

- rappel (cours 1) : un LLM a une fenêtre de contexte finie (par exemple 200 000 *tokens*)
- les longues conversations, les gros dépôts de code, les tâches sur plusieurs fichiers la dépassent
- pour un agent, c'est pire : chaque résultat d'outil (un fichier lu, la sortie de `pytest`) s'empile dans le contexte
- **le contexte *est* la mémoire de travail de l'agent**

La compaction : résumer pour tenir dans la fenêtre

- le *harness* résume automatiquement les parties anciennes de la conversation, pour libérer du contexte
- la stratégie de base : *demander au modèle* de résumer l'historique, puis garder le résumé et les tours récents



- la compression est **avec perte** : des détails du début peuvent disparaître
- si l'agent semble avoir oublié une consigne donnée plus tôt, le contexte a probablement été compacté

- **des *prompts* ciblés** : ne pas déverser des fichiers entiers, les désigner par leur chemin

- **des *prompts* ciblés** : ne pas déverser des fichiers entiers, les désigner par leur chemin
- **une exploration progressive** : lire les fichiers au besoin, pas tout le dépôt d'un coup

- **des *prompts* ciblés** : ne pas déverser des fichiers entiers, les désigner par leur chemin
- **une exploration progressive** : lire les fichiers au besoin, pas tout le dépôt d'un coup
- **repartir de zéro quand il le faut** : une session neuve vaut mieux qu'une longue session compactée (cours 2)

- **des *prompts* ciblés** : ne pas déverser des fichiers entiers, les désigner par leur chemin
- **une exploration progressive** : lire les fichiers au besoin, pas tout le dépôt d'un coup
- **repartir de zéro quand il le faut** : une session neuve vaut mieux qu'une longue session compactée (cours 2)
- **explorer avec les outils** : laisser l'agent découvrir les fichiers plutôt que les lui fournir d'avance
  c'est le RAG du cours 2, fait par l'agent lui-même : `grep` et la lecture de fichiers sont son étape *Retrieve*

- **des *prompts* ciblés** : ne pas déverser des fichiers entiers, les désigner par leur chemin
- **une exploration progressive** : lire les fichiers au besoin, pas tout le dépôt d'un coup
- **repartir de zéro quand il le faut** : une session neuve vaut mieux qu'une longue session compactée (cours 2)
- **explorer avec les outils** : laisser l'agent découvrir les fichiers plutôt que les lui fournir d'avance
  c'est le RAG du cours 2, fait par l'agent lui-même : `grep` et la lecture de fichiers sont son étape *Retrieve*
- `AGENTS.md` : le contexte du projet, qui survit à la compaction (voir section suivante)

1. De l'assistant à l'agent
2. La boucle agentique
3. Du *harness* à l'agent de code
4. Rappels sur le développement logiciel
5. Le développement agentique : autonomie et pratique
6. Gérer la fenêtre de contexte
7. Étendre et guider l'agent : AGENTS.md et *skills*

- un fichier Markdown à la racine du dépôt [†]

†. *AGENTS.md : a simple, open format for guiding coding agents*↗. agents.md. 2025.

AGENTS.md : la mémoire du projet

- un fichier Markdown à la racine du dépôt [†]
- il survit à la compaction, parce qu'il est **relu** par le *harness* et envoyé au modèle **à chaque tour**

†. *AGENTS.md : a simple, open format for guiding coding agents*↗. agents.md. 2025.

AGENTS.md : la mémoire du projet

- un fichier Markdown à la racine du dépôt [†]
- il survit à la compaction, parce qu'il est **relu** par le *harness* et envoyé au modèle **à chaque tour**
- son contenu : comment construire et tester, les conventions, l'architecture, ce qu'il ne faut *pas* toucher

†. *AGENTS.md : a simple, open format for guiding coding agents*↗. agents.md. 2025.

AGENTS.md : la mémoire du projet

- un fichier Markdown à la racine du dépôt [†]
- il survit à la compaction, parce qu'il est **relu** par le *harness* et envoyé au modèle **à chaque tour**
- son contenu : comment construire et tester, les conventions, l'architecture, ce qu'il ne faut *pas* toucher
- court et factuel : chaque ligne coûte du contexte ; on le versionne et on le relit, comme du code

†. *AGENTS.md : a simple, open format for guiding coding agents*↗. agents.md. 2025.

AGENTS.md : la mémoire du projet

- un fichier Markdown à la racine du dépôt †
- il survit à la compaction, parce qu'il est **relu** par le *harness* et envoyé au modèle **à chaque tour**
- son contenu : comment construire et tester, les conventions, l'architecture, ce qu'il ne faut *pas* toucher
- court et factuel : chaque ligne coûte du contexte ; on le versionne et on le relit, comme du code
- l'équivalent, pour le web, de `robots.txt`, qui dit aux robots d'indexation ce qu'ils peuvent visiter

un exemple

```
# AGENTS.md
## Conventions
- Java 21
- Javadoc on every public method
## Commands
- build: ‘mvn -q package‘
- test: ‘mvn -q test‘
## Do not
- edit generated code
- commit secrets
- push to main
```

†. *AGENTS.md : a simple, open format for guiding coding agents*↗. agents.md. 2025.

Les *skills* : du savoir-faire en paquet

- le cours 2 citait les *skills* comme troisième source d'outils ; les voici [†]

†. Anthropic. *Agent Skills*↗. agentskills.io. 2025.

Les *skills* : du savoir-faire en paquet

- le cours 2 citait les *skills* comme troisième source d'outils ; les voici [†]
- une ***skill*** = un **dossier**
  - un fichier `SKILL.md`
  - éventuellement des scripts et des fichiers de référence

†. Anthropic. *Agent Skills*↗. agentskills.io. 2025.

Les *skills* : du savoir-faire en paquet

- le cours 2 citait les *skills* comme troisième source d'outils ; les voici [†]
- une ***skill*** = un **dossier**
  - un fichier `SKILL.md`
  - éventuellement des scripts et des fichiers de référence
- `SKILL.md` = un **en-tête** (un nom, une description d'une ligne), puis des **instructions**

†. Anthropic. *Agent Skills*↗. agentskills.io. 2025.

Les *skills* : du savoir-faire en paquet

- le cours 2 citait les *skills* comme troisième source d'outils ; les voici †
- une ***skill*** = un **dossier**
  - un fichier `SKILL.md`
  - éventuellement des scripts et des fichiers de référence
- `SKILL.md` = un **en-tête** (un nom, une description d'une ligne), puis des **instructions**
- `AGENTS.md` dit *comment marche ce projet* ; une *skill* dit *comment faire un type de tâche*, d'un projet à l'autre

un dossier de *skill*

```
skills/
  python-unit-tests/
    SKILL.md
    run_tests.py
```

`SKILL.md`

```
---
name: python-unit-tests
description: Run the unit tests
  of a Python project.
---
# Instructions
1. One test file per module,
   in tests/.
2. Run run_tests.py until
   all tests pass.
```

†. Anthropic. *Agent Skills*↗. agentskills.io. 2025.

Charger une *skill* à la volée

le contexte

system prompt, `AGENTS.md`

sur le disque

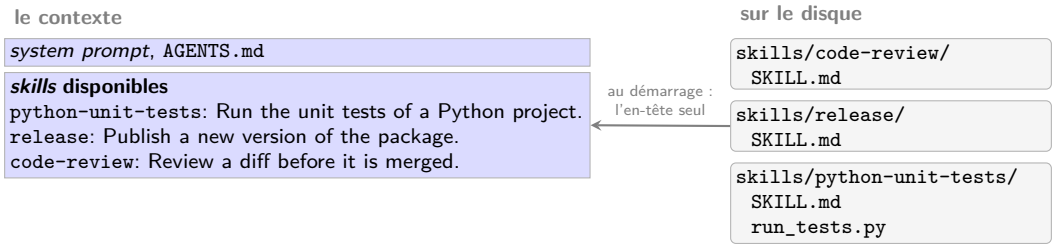
```
skills/code-review/
  SKILL.md
```

```
skills/release/
  SKILL.md
```

```
skills/python-unit-tests/
  SKILL.md
  run_tests.py
```

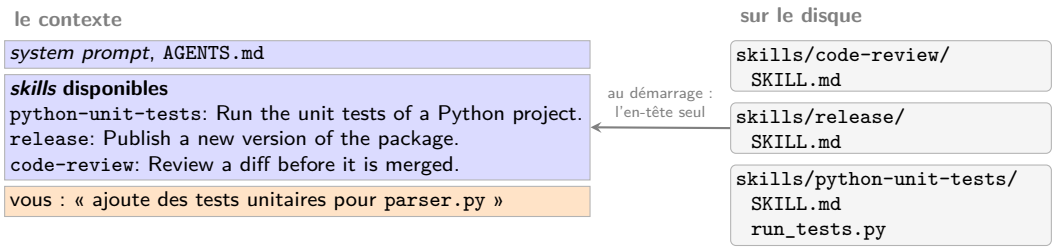
au départ, le contexte contient le *system prompt* et `AGENTS.md`

Charger une *skill* à la volée



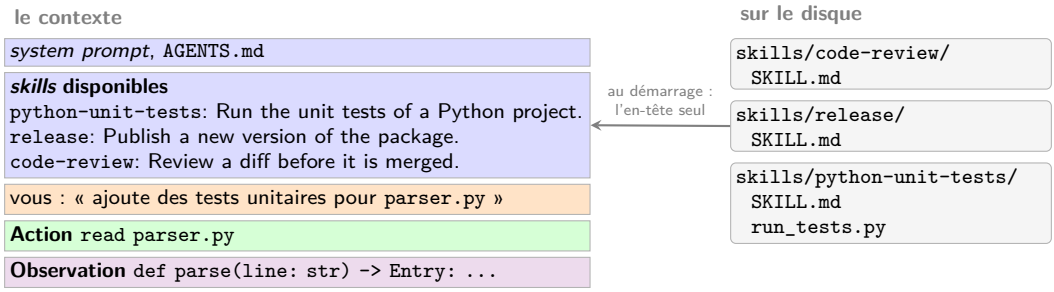
au démarrage, le *harness* ne lit que l'**en-tête** de chaque SKILL.md : une ligne par *skill*

Charger une *skill* à la volée



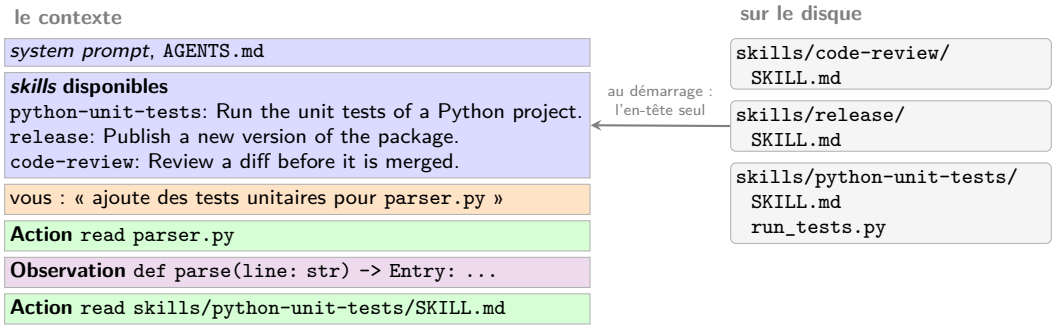
la tâche arrive

Charger une *skill* à la volée



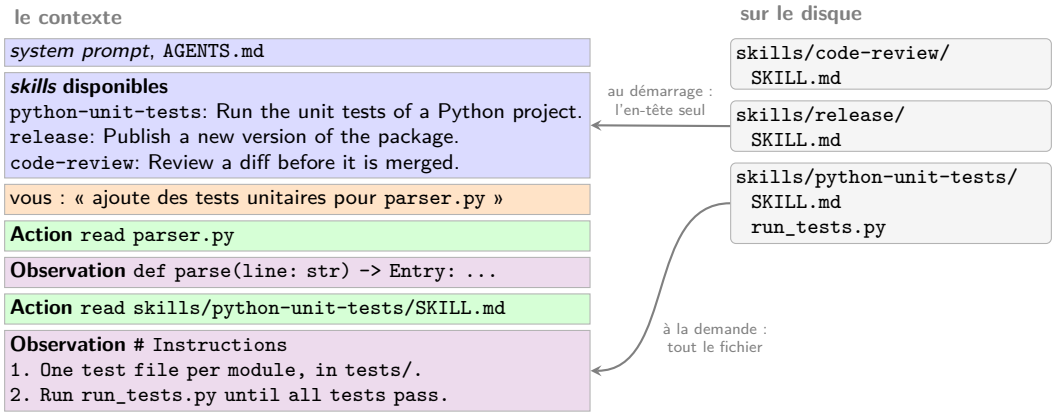
le modèle commence par lire le **code à tester**

Charger une *skill* à la volée



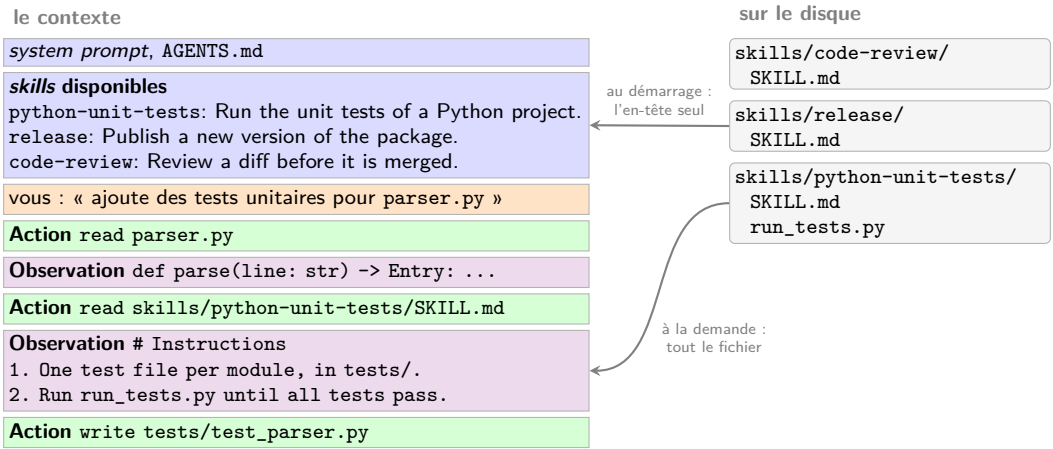
puis il reconnaît une description qui correspond à la tâche, et lit le `SKILL.md` avec un outil

Charger une *skill* à la volée



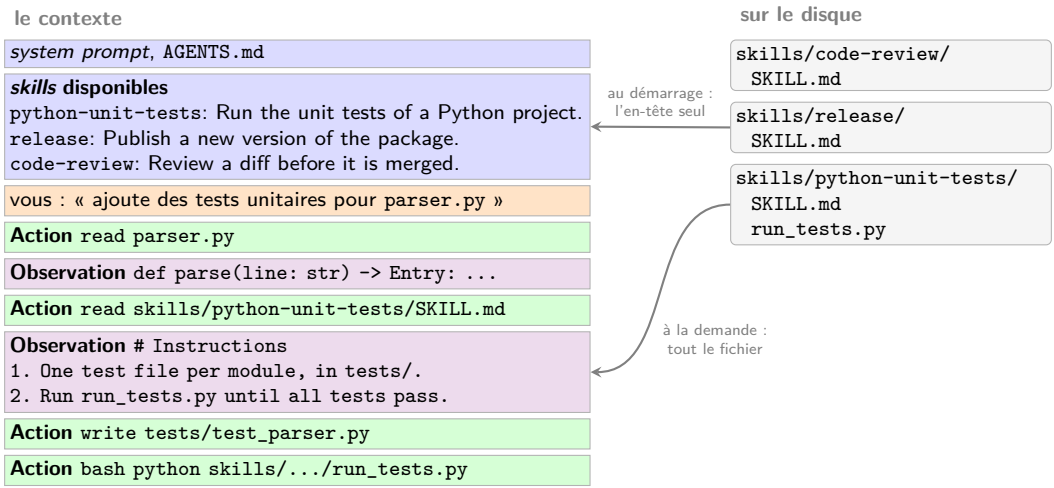
les instructions entrent dans le contexte **au moment où elles servent**

Charger une *skill* à la volée



l'agent écrit les tests en suivant les instructions : un fichier par module, dans `tests/`

Charger une *skill* à la volée



le script de la *skill* est **exécuté** (pas lu)

- un format ouvert : les agents les cherchent dans un dossier du projet, `.agents/skills/` ; une *skill* écrite pour un agent marche avec les autres

Où trouver des *skills*

- un format ouvert : les agents les cherchent dans un dossier du projet, `.agents/skills/` ; une *skill* écrite pour un agent marche avec les autres

`skills.sh`↗	un annuaire ; `npx skills add <dépôt>` installe une *skill*
`VoltAgent/awesome-agent-skills`↗	plus de 1000 *skills*, officielles et communautaires
`obra/superpowers`↗	une méthode de développement en *skills* (plan, TDD…)
`anthropics/skills`↗	les *skills* de référence d'Anthropic

Où trouver des *skills*

- un format ouvert : les agents les cherchent dans un dossier du projet, `.agents/skills/` ; une *skill* écrite pour un agent marche avec les autres

`skills.sh`↗	un annuaire ; `npx skills add <dépôt>` installe une *skill*
`VoltAgent/awesome-agent-skills`↗	plus de 1000 *skills*, officielles et communautaires
`obra/superpowers`↗	une méthode de développement en *skills* (plan, TDD…)
`anthropics/skills`↗	les *skills* de référence d'Anthropic

- une *skill* tierce reste une dépendance : **on la lit avant de l'installer**

Où trouver des *skills*

- un format ouvert : les agents les cherchent dans un dossier du projet, `.agents/skills/` ; une *skill* écrite pour un agent marche avec les autres

skil
Volt
obra
anth

#	SKILL	8W ACTIVITY	INSTALLS
1	**find-skills** vercel-labs/skills		4.2M
2	**grill-me** mattpocock/skills		1.5M
3	**hyperframes-cli** heygen-com/hyperframes		1.4M
4	**hyperframes** heygen-com/hyperframes		1.4M

les *skills* les plus installées sur `skills.sh`

- un

À retenir

1. **un agent = un LLM + des outils + un *harness*** : c'est le *harness*, pas le modèle, qui décide du comportement de l'agent
2. **la boucle fait la puissance de l'agent, et ses pannes** : le raisonnement ReAct, les conditions d'arrêt, la détection des blocages
3. **l'autonomie est un cadran** : plan, laisse courte, longue itération, YOLO ; le régler selon le rayon d'impact et la vérifiabilité
4. **le contexte est fini, et la compaction est avec perte** : des *prompts* ciblés, `AGENTS.md`, repartir de zéro quand il le faut
5. **les *skills* étendent le savoir-faire de l'agent** ; `AGENTS.md` le garde sur les rails
6. **travailler en sécurité** : isoler l'agent, aucun secret à sa portée, et tout ce qu'il lit est une donnée non fiable
7. **vous êtes responsable** : l'IA accélère le développement, elle ne transfère pas la responsabilité

- vous savez maintenant *utiliser* un agent, et en garder le contrôle
- cours 4 : **plusieurs agents, et l'avenir du développement logiciel**
  - un QCM aura lieu lors de cette séance
- de délégant à *concepteur*

- vous savez maintenant *utiliser* un agent, et en garder le contrôle
- cours 4 : **plusieurs agents, et l'avenir du développement logiciel**
  - un QCM aura lieu lors de cette séance
- de délégant à *concepteur*



À ne pas manquer :
la conférence de Masoud Saeida Ardekani

Staff Software Engineer, Google (Mountain View, CA)

le 9 novembre, à 18h, amphi Étoile

Quiz en ligne

- répondez au quiz du cours 3, depuis votre téléphone ou votre ordinateur
- scannez le QR code, ou allez sur le lien ci-dessous



https://digistorm.app/p/5583387